Functional Description of
Diagnosis on

CAN

For

ABS10.3ME Platform Project

System: **Customer:** **Vehicle:**	Front/Rear Brake Split; 2 Channel ABS10.3 ME Full Integral System 2 Channel ABS10.3 ME Semi Integral System	**Equipment:** **Tester/Diagnosis:**
Distribution:	Customer, Internal	

Approval:

Signature	Department
Liersch Geoff	2WP/P 2WP/NE
Fellmeth Martin Reiner	2WP/ENG 2WP/EYS
Ono Shunsaku	2WP/ESS 2WP/ESS3 2WP/ESS4
Adachi Eiji	2WP/ESS2 2WP/ESS5
Deepak Sanda Nagaraja Rao	RBEI/ESB2
Venkatasivaprasad Reddy Chintham	RBEI/ESB2
Vibha Hanumantharaya	RBEI/ESB2

Please refer signature workflow.

History

Version No.	Date	Editor	Reviewer	Description of changes
2.0	04/06/2019	Ashley	Nikhiladheesh Yogish	- Based on **0265T81430_01** - Updated Section 5.2 to include sub-functions supporting supressPositiveResponse bit - Updated services with subfunction supporting supressPositiveResponse bit (Section 6,7,9,10,13) - Removed eCBS valves from Section 18 and from section 11.1 (ECU Input Signal). - Updated definition of DTCStatusAvailability Mask from 0xFFH to 0x2FH - Note added in Section 8 to indicate that a persistent failure will remain logged even after ClearDiagnosticInformation request is given - Updated "Definition of SubFunction" table in section 14 - Removed write service for Reprogramming Date and SW Fingerprint - Updated Section 11 to include notes to indicate the NRC handling adopted when the PDM location is empty. - Updated section 11.17 to indicate the responses obtained for the Diagnostic Active Session read service - Updated section 11.7 SWVersion from a 4 to 2 byte read - Section 13.2 has been updated to include note regarding the loop repetition strategy followed
	20/07/2020	Ashley		- Section 13.4 WSS Request Routine Results: Resolution of speed has been changed to 0.05625kmph from 1kmph to be in-line with the previous Generation requirements.
	21/01/2021	Vibha H		- Section 5.1 Updated the list of services availability based on session - Section 17.3.3 Memory check routine response code is updated - Section 17.3.5 Check Programming dependency response code is updated for signature check.

Contents

1 Referenced documents 6
2 Introduction 6
3 Communication via CAN 6
4 Service Overview 14
5 Availability of Services: Based on Session 15
5.1 List of service availability based on the diagnostic session and security 15
5.2 List of the supported sub-function 17
6 DiagnosticSessionControl 19
6.1 Negative Response 22
7 ECU Reset 23
7.1 Negative Response 24
8 Clear Diagnostic Information 25
8.1 Negative Response 26
9 Read DTC information 27
9.1 reportNumberOfDTCByStatusMask 28
9.2 reportDTCbyStatusMask 29
9.3 Negative Response 30
10 Security access 31
10.1 Request Seed 31
10.2 Send Key 32
10.3 Negative Response 34
11 ReadDataByIdentifier 35
11.1 ECU Input signal 36
11.2 Variant Data (Bike Model) 37
11.3 Pressure Sensor Signals 38
11.4 Reprogramming Date 39
11.5 ProcessByte 39
11.6 Customer part number 40
11.7 SW version 40
11.8 SW Fingerprint 41

11.9 Vehicle Identification number 42
11.10 EOL-Lean Angle Data 43
11.11 EOL-Pitch Angle Data 44
11.12 EOL Mode 45
11.13 IMU Sensor Status 46
11.14 EOL Expected Lean Angle 47
11.15 EOL Expected Pitch Angle 47
11.16 BB Number 48
11.17 Diagnostic Active Session 48
11.19 Negative Response 49
12 WriteDataByIdentifier 50
12.1 ProcessByte 51
12.2 Vehicle Identification number 52
12.3 EOL Mode 53
12.4 EOL Expected Lean Angle 54
12.5 EOL Expected Pitch Angle 55
12.6 Variant Data(Bike Model) 56
12.7 Negative Response 57
13 Routine Control 58
13.1 Evac and Fill 59
13.2 Repair Bleeding 63
13.3 Actuator Test 67
13.4 Wheel Speed Sensor Test 72
13.5 ABS SILA Blinking Test 76
13.6 Sensor Mounting Validation Test 78
13.7 Negative response 80
14 Communication Control 81
14.1 Negative response 83
15 ControlDTCSetting 84
15.1 Negative response 85
16 Tester Present 86
16.1 Negative Response 87

17 Flash Download 88
17.1 Erasure Routines 88
17.2 Flash Routine 91
17.2.1 Request Download 91
17.3 Transfer Data 92
17.3.1 Request Transfer Exit 94
17.3.2 Memory Check 96
17.3.3 Memory Check Routine 96
17.3.4 Memory Check Routine Negative Response 97
17.3.5 Check Programming Dependency 99
17.4 Flashing Sequences 102
18 Appendix A: List of Actuators 103

1 Referenced documents

International documents

The general requirements of diagnosis communication described in this document are based on the Unified diagnostic services (UDS) design standards published by ISO (International Organization for Standardization) with the restrictions / additions as defined by the subsequent sub-sections:

ISO 14229 : Road Vehicles – Diagnostic Systems: Diagnostic Services Specification
ISO 15765 : Road Vehicles- Diagnostic communication over Controller Area Network
Customer documents

2 Introduction

UDS based CAN diagnosis provides the capability of quickly accessing the information and functions of the ECU over the CAN, without disconnecting the cable harness. This diagnosis provides services to:

- Identify the ECU
- Read input signals applied to the ECU
- Actuate all switching elements connected to the ECU
- Read out internal information, such as the contents of the fault memory etc, stored in the ECU
- Execute ROM resident test sequence routines

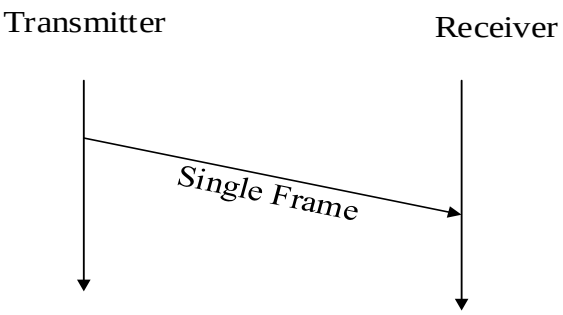
3 Communication via CAN

Data stream

The ECU supports CAN diagnostic messages using the Unacknowledged Segmented Data Transfer (USDT) mechanism as specified in ISO 15765-2. Accordingly the following two types of CAN data streams are supported:

a) Non-segmented data stream :

Consists of a Single Frame (SF) of non-segmented CAN message. This can be used for messages with maximum data length of 7 data bytes



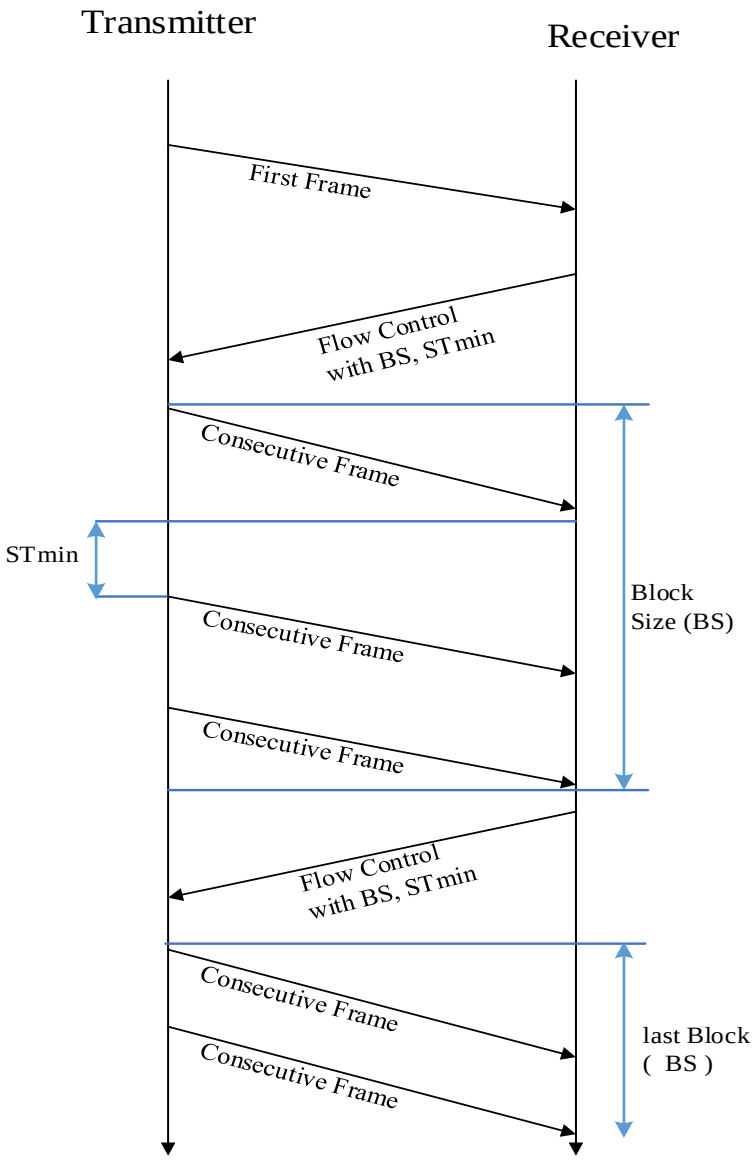
b) Segmented data stream

Consists of multiple frames of segmented CAN messages. This can be used when the maximum data length exceeds 7 data bytes.

The data stream comprises of a First Frame (FF) and a number of Consecutive Frames (CF) containing the subsequent data bytes that follow the First Frame. A Handshake, in the form of a Flow Control (FC), has to be send by the receiver node after reception of the First Frame.

The Flow Control should contain information regarding the maximum number of Consecutive Frames the receiver can allow (Block Size –BS) after which the sender has to wait for an authorisation from the receiver to continue, and the minimum time the sender has to wait between transmission of two consecutive frames (Separation Time -STmin).

Note that functional addressing does not support segmented data streams.


Transmitter
Receiver
First Frame
Flow Control
with BS, STmin
Consecutive Frame
STmin
Block
Size (BS)
Consecutive Frame
Consecutive Frame
Flow Control
with BS, STmin
Consecutive Frame
Consecutive Frame
last Block
(BS)

Message Types

Each of the CAN message types shown above .i.e Single Frame (SF), First Frame (FF), Consecutive Frame (CF) and Flow Control (FC) contain two main components, namely :

- Protocol Control Information (PCI):
  Indicates the frame type, sequence number counter for multiple frames, data length, Separation time (STmin), Block Size (BS) and Flow Status (FS).

- Transmission data (Data): Contains the data to be transmitted

PCI and Transmission Data together make up the Protocol Data unit (PDU).

Data Field Structure					
FRAME TYPE	**BYTE 1**		**BYTE 2**	**BYTE 3**	**BYTES 4-8**
	Bits 7-4	Bits 3-0			
SINGLE FRAME (SF)	**0000**	**SF_DL**	DB 1	DB 2	DB 3 – 7
FIRST FRAME (FF)	**0001**	**FF_DL**		DB 1	DB 2 – 6
CONSECUTIVE FRAME (CF)	**0010**	**SN**	DB 1	DB 2	DB 3 – 7
FLOW CONTROL (FC)	**0011**	**FS**	BS	STmin	00

SF_DL : 4 Bit Single Frame Data length
FF_DL : 12 Bit First Frame Data length
SN : Sequence Number
FS : Status of data stream
BS : Block Size
STmin : Minimum Separation Time
DB : Data byte

Single Frame

Single Frame message type is used for data blocks with a maximum data length of 7 Bytes and is identified by the upper nibble of PCI set to 0000B.

Definition of Single Frame data length (SF_DL):

Hex Value	Description
0 - 7	Single Frame Data length (SF_DL) is a 4 bit value that contains the number of data bytes present in the single frame

First Frame

First Frame message type is used for data blocks which have more than 7 data Bytes. They are used in combination with one or more Consecutive Frames and is identified by the upper nibble of PCI set to 0001B.

Definition of First Frame Data length (FF_DL):

Hex Value	Description
000 – 007	Reserved
008 – FFF	First Frame Data length (FF_DL) contains number of data bytes.

Consecutive Frame

Consecutive Frames follows the First Frame and is identified by the upper nibble of PCI set to 0010B.

The lower nibble of the PCI has a Sequence Number (SN) which is determined as follows:

First Frame is regarded as containing Sequence Number 0.Hence the first Consecutive Frame always holds Sequence Number 1.Sequence Number is incremented by one for every additional Consecutive Frame. A Flow Control frame sent ‘in between’ does not affect the Sequence Number counting. When the Sequence Number reaches the value 15, the following Consecutive Frame will be sent with Sequence Number set to 0.

Definition of Sequence Number (SN):

Protocol Data name	FF	CF	CF	CF	CF	CF	CF	CF	CF
SN (hex)	0	1	2	...	E	F	0	1	...

Hex Value	Description
0 – F	Sequence Number (SN) The Sequence Number has a range of 0 to 15. SN-Value is stored in the lower 4 bits of Byte 1.

Flow Control

After the First Frame or the last Consecutive Frame of a block of frames (when further CF's are required to complete the data transfer), a Flow Control message must be sent by the receiver node, and is identified by the upper nibble of PCI set to 0011B. With the Flow Control, the receiving node notifies the transmitting node how many messages it can receive between two Flow Control frames (maximum Block Size, Bsmax) and at what rate between Consecutive Frames (minimum Separation Time, STmin). The lower nibble of the Flow Control PCI shall have a Flow Control Status (FS).

Definition of Flow Control Status (FS):

Hex Value	Description
0	Continue To Send (CTS) The Receiver is ready and is waiting for maximally BS Consecutive Frames
1	Wait (WT) The receiver is not ready to receive new messages and informs sender to wait for new FlowControl and to restart its N_BS timer. If FlowStatus is set to Wait, the values of BS and STmin in the FlowControl message are not relevant and shall be ignored.
2	Overflow (OVFLW) The FF_DL received in FF is greater than the buffer size then FS is set to overflow. If FlowStatus is set to Overflow, the values of BS and STmin in the FlowControl message are not relevant and shall be ignored.
3 - F	Reserved

Block Size (Bsmax):

Hex Value	Description
00	Block Size (Bsmax) Indicates that no more Flow Control frames are allowed during transmission of the segmented data frame. All Consecutive Frames will be sent without any stop for further Flow Control frames.
01 – FF	Block Size (Bsmax) Defines the maximum amount of Consecutive Frames that can be received without an intermediate Flow Control frame.

Separation Time (STmin):

Hex Value	Description
00 – FF	Separation Time (STmin) The Separation Time defines the minimal allowed time gap (in ms) between the transmissions of Consecutive Frames.

Communication timing

Parameter of the ECU:

Only one tester address is permitted in one diagnosis session all the time. A change of the tester address causes an ECU Timeout. The number of switches between functional and physical addressing is unlimited in one diagnosis session.

Name	Category	Value	Description
Bus type	Communication	CAN	Type of transport media
Baud rate	Communication	500000	Bus Speed of the used transport media.
Addressing Scheme	Communication	Normal	Addressing Scheme used for physical/Functional requests and responses.
CAN-ID Type	Communication	11-bit	CAN-ID Type used for physical/Functional requests and responses
Max Length of TP Message	Communication	4095	
Physical Request CAN-ID	Communication	0x7E1	The request CAN identifier for physical requests. The hex value not only describes the identifier but also the priority of the message. The higher the number, the lower the priority.
Response CAN-ID	Communication	0x7E9	The response CAN identifier for physical responses. The response for functional requests is sent via the physical path.
Functional Request CAN-ID	Communication	0x7DF	The request CAN identifier for functional requests.
P2Server	Timing	50ms	Performance requirement for the server to start with the response message after the reception of a request message.
P2*Server	Timing	5000ms	Performance requirement for the server to start with the response message after the transmission of a negative response message with response code 78 hex (enhanced response timing).
S3Server	Timing	5000ms	Time for the server to keep a diagnostic session other than the Default Session active while not receiving any diagnostic request message
STmin	Timing	127ms	(Separation Time) defines the minimum time gap between consecutive frames.
Blocksize	Communication	0	The number 0 tells the sender, that no more flow controls should disrupt the sending of the remaining flow controls
Timeout As	Timing	1000ms	Time for transmission of frame, sender side
Timeout Ar	Timing	1000ms	Time for transmission of frame, receiver side
Timeout Bs	Timing	1000ms	Time until reception of next flow control
Time Br	Timing	10ms	Time for next transmission of flow control
Time Cs	Timing	10ms	Time until next transmission of consecutive frame
Timeout Cr	Timing	1000ms	Time until reception of next consecutive frame
RC78CompletionTimeout	Timing	50000ms	Time period the tester accepts repeated negative responses with response code 0x78 and waits for a positive response further on. After sending 10 consecutive NRC78, Dcm sends NRC10 (generalReject).

4 Service Overview

Overview of project specific supported services and functions

The UDS services / functions supported by the ECU, together with the request code from tester and the corresponding response code from the ECU are listed below:

SID	Services	Addressing Mode		Positive Response
		Physical	Functional	
$10	DiagnosticSessionControl	x	x	$50
$11	ECUReset	x	-	$51
$14	ClearDiagnosticInformation	x	-	$54
$19	ReadDTCInformation	x	-	$59
$22	ReadDataByIdentifier	x	-	$62
$27	SecurityAccess	x	-	$67
$2E	WriteDataByIdentifier	x	-	$6E
$31	RoutineControl	x	-	$71
$28	CommunicationControl	x	x	$68
$85	ControlDTCSetting	x	x	$C5
$3E	TesterPresent	x	x	$7E
$34	RequestDownload	x	-	$74
$36	TransferData	x	-	$76
$37	RequestTransferExit	x	-	$77

Note:
x : Service is available in the specified addressing mode
- : Service is unavailable in the specified addressing mode

5 Availability of Services: Based on Session

The ECU will execute a UDS service requested by the Tester depending on:

- The type of Diagnostic mode active at the time of request
- Vehicle speed at the time of request
- ECU being busy executing some other internal routine (eg: executing control sequence)

5.1 List of service availability based on the diagnostic session and security

Services	Default	Extended	Programming
DiagnosticSessionControl			
Default Session	Y	Y	Y
Programming Session	N	Y	Y
Extended Session	Y	Y	Y
EcuReset			
Hard Reset	N	Y	Y
Clear All DTC	Y	Y	N
ReadDtcInformation			
reportNumberOfDTCByStatusMask	Y	Y	N
reportDTCByStatusMask	Y	Y	N
SecurityAccess			
RequestSeed	Y	Y	Y
SendKey	Y	Y	Y
ReadDataByIdentifier			
ECU input signal (1)	Y	Y	N
Variant Data	Y	Y	N
Pressure Sensor Signals	Y	Y	N
Reprogramming Date	Y	Y	Y
Process Byte	Y	Y	N
Customer Part Number	Y	Y	Y
SW Version	Y	Y	N
SW Fingerprint	Y	Y	Y
Vehicle Identification Number(VIN)	Y	Y	N
EOL-Lean Angle Data	Y	Y	N
EOL-Pitch Angle Data	Y	Y	N
EOL Mode	Y	Y	N
IMU Sensor Status	Y	Y	N
EOL Expected Lean Angle	Y	Y	N
EOL Expected Pitch Angle	Y	Y	N
BBNumber	Y	Y	N
Diagnostic Active Session	Y	Y	Y
WriteDataByIdentifier			
ProcessByte	Y+	Y+	N
Vehicle Identification Number(VIN)	Y+	Y+	N
EOL Mode	Y+	Y+	N
EOL Expected Lean Angle	Y+	Y+	N
EOL Expected Pitch Angle	Y+	Y+	N
Variant Data	Y+	Y+	N
Routine Control			

Evac and Fill	N	Y+	N
Repair Bleed	N	Y+	N
Actuator test	N	Y+	N
Wheel speed sensor test	N	Y+	N
ABS Sila Lamp blinking test	N	Y+	N
Senor Mounting Validation Test	N	Y+	N
Erase Routine	N	N	Y
Memory check	N	N	Y
Check programming dependency	N	N	Y
Communication Control	N	Y	N
Control DTC Setting	N	Y	N
Tester Present	Y1	Y	Y
Request Download	N	N	Y
Transfer Data	N	N	Y
Request Transfer Exit	N	N	Y

Note:

Y: The service can be executed in this session
N: The service cannot be executed in this session
Y+: Protected services/function.
Y1: The service is supported in this session without function i.e. session timer inactive

5.2 List of the supported sub-function

Service		Description
SID (hex)	**Sub-function (hex)**	
10		**DiagnosticSessionControl**
	01/81	DefaultSession
	02/82	ProgrammingSession
	03/83	ExtendedSession
11		**ECUReset**
	01/81	Hard Reset
19		**ReadDTCInformation**
	01/81	reportNumberOfDTCByStatusMask
	02/82	reportDTCByStatusMask
27		**SecurityAccess**
	01/81	RequestSeed
	02/82	SendKey
28		**CommunicationControl**
	00/80	EnableRxAndTx
	01/81	EnableRxAndDisableTx
31		**RoutineControl**
	01/81	StartRoutine
	02/82	StopRoutine
	03/83	RequestRoutineResults
85		**ControlDTCSetting**
	01/81	DTC storage enabled
	02/82	DTC storage disabled
3E		**TesterPresent**
	00/80	suppressPosRspMsgIndicationBit

SuppressPositiveResponse bit:

The most significant bit (MSBit) in the sub-function byte of certain client (e.g., tester) requests, is used to activate and deactivate the positive server response. This bit is called the SuppressPositiveResponse bit (SPR) and does not belong to the sub-function. Therefore it is not contained in the positive response from the server. Below the definition of the SuppressPosResponse:

SPR	Description
0b	The server (e.g., ECU) must send a positive or negative response.
1b	The server (e.g., ECU) must suppress the positive response. A positive response may be sent only if a negative response with NRC 0x78 has been sent beforehand.

6 DiagnosticSessionControl

The DiagnosticSessionControl is used to enable different diagnostic sessions in the ECU. The current project supports the following sessions:

Default Session

This mode does not permit the execution of all diagnostic services possible for the ECU (see "Availability of Services" section for services supported during this session).
The ECU shall always start in default session with only one session being active at any given time. The ECU should only transition to a non-default session following a request from the diagnostic tester to do so.

Extended Diagnostic session

During this mode the ECU will be switched to Extended Diagnostic session and all the controller functions will be set to passive with only the diagnostic services set to active. All lamps will be turned ON to indicate the non-availability of the controller.

During the Extended Diagnostic session, the ECU will monitor the time taken for every new tester request. The ECU will return to Normal / Default Diagnosis mode if this time exceeds S3Server (see "Communication Parameters" section mentioned above). The extended session can be entered only if vehicle speed is less than 10kmph.

Programming Session

If the reprogramming of the ECU is needed, it should be set in this mode.

DiagnosticSessionControl Request:

Tester ⇒ ECU : StartDiagnosticSessionRequest		
Byte	Contents	Value
1	DiagnosticSessionControl	10 H
2	Sub-function	xx H

Definition of Sub-function:

No	Sub-function	Session Value
1	Default Session	01 H / 81 H
2	Programming Session	02 H / 82 H
3	ExtendedDiagnosticSession	03 H / 83 H

Default Session Positive Response:

If the ECU is currently in the default session (as is the case initially), the following positive response is sent:

ECU ⇒ Tester: StartDiagnosticSessionPositiveResponse		
Byte	Contents	Value
1	StartDiagnosticSessionPR	50 H
2	DefaultSession	01 H
3	P2Server_max Timing HB	xx H
4	P2Server_max Timing LB	xx H
5	P2*Server_max Timing HB	xx H
6	P2*Server_max Timing LB	xx H

Programming Session Positive Response:

The ECU send below response when request for ECU Programming Mode is made. This positive response indicates that the software is now in the bootblock.

ECU ⇒ Tester: StartDiagnosticSessionPositiveResponse		
Byte	Contents	Value
1	StartDiagnosticSessionPR	50 H
2	Programming Session	02 H
3	P2Server_max Timing HB	xx H
4	P2Server_max Timing LB	xx H
5	P2*Server_max Timing HB	xx H
6	P2*Server_max Timing LB	xx H

Extended Diagnostic Session Positive Response:

The ExtendedDiagnosticSession permit the execution of all diagnostic services possible for the ECU. If a current diagnostic session has been requested that is already active, the ECU will respond with a positive response and shall not have any effect on the current session or its available services and/or functionality.

ECU ⇒ Tester: StartDiagnosticSessionPositiveResponse		
Byte	Contents	Value
1	StartDiagnosticSessionPR	50 H
2	ExtendedDiagnosticSession	03 H
3	P2Server_max Timing HB	xx H
4	P2Server_max Timing LB	xx H
5	P2*Server_max Timing HB	xx H
6	P2*Server_max Timing LB	xx H

Note:
1. P2Server_max is in 1 ms resolution
   P2*Server_max is in 10ms resolution
2. If the suppressPosRspMsgIndicationBit is set then the positive response from the ECU is suppressed.

6.1 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: DiagnosticSessionControlNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	DiagnosticSessionControl	10 H
3	NegativeResponseCode	xx H

The negative response codes (NRC) supported for the DiagnosticSessionControl are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	Sub-FunctionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.
0x22	ConditionsNotCorrect	This NRC shall be returned if the criteria for the request DiagnosticSessionControl are not met.
0x78	requestCorrectlyReceived-ResponsePending	This NRC indicates that the request message was received correctly, but the action to be performed is not yet completed and the server is not yet ready to receive another request. As soon as the requested service has been completed, the server shall send a positive response message or negative response message with a response code different from this.
0x88	VehicleSpeedTooHigh	This NRC indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met.

7 ECU Reset

This service requests the server to effectively perform a server reset i.e.re-initialization of volatile and non-volatile memory .The ECUReset positive response message shall be sent before the reset is executed in the server.

The ECUReset can be done only at vehicle speed less than 10 km/h. Otherwise, the ECU responds with a negative response code of 88H indicating vehicleSpeedTooHigh.
In boot block there will not be any speed check for this service.

ECU Reset Request:

Tester ⇒ ECU: ECUResetRequest		
Byte	Contents	Value
1	ECUReset	11 H
2	SubFunction	xx H

After a successful server reset the server shall activate the default Session. During this time the ECU does not accept any request messages nor sends any response messages.

Definition of Sub function:

No	ECU Reset type	Session Value
1	Hard Reset	01 H / 81 H

Hard Reset Positive Response:

ECU will send the following Positive response after successful Hard Reset.

Tester ⇒ ECU : ECU Reset PositveResponse		
Byte	Contents	Value
1	ECUResetPR	51 H
2	Hard Reset	01 H

7.1 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: ECU Reset NegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	ECU Reset	11 H
3	Response Code	xx H

The negative response codes (NRC) supported for the ECU Reset are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	Sub-functionNotSupported	This NRC shall be sent if the sub function parameter is not supported.
0x88	vehicleSpeedTooHigh	This NRC indicates that the requested action will not be taken because the server prerequisite condition for vehicle speed is not met (current vehicle speed is above a pre-programmed maximum threshold).

8 Clear Diagnostic Information

The "ClearDiagnosticInformation" service will clear the contents of all failure memory stored in the EEPROM of the ECU. The server shall send a positive response even if no DTCs are stored.

Clear Diagnostic Information Request:

Tester ⇒ ECU : ClearDiagnosticInformationRequest		
Byte	Contents	Value
1	ClearDiagnosticInformation	14 H
2	GroupOfDTC High Byte	xx H
3	GroupOfDTC Mid Byte	xx H
4	GroupOfDTC Low Byte	xx H

Definition of GroupOfDTC:

No	DID Description	DID value
1	All DTCs	GroupOfDTC High Byte : FF H GroupOfDTC Mid Byte : FF H GroupOfDTC Low Byte : FF H

Positive Response:

On gaining access to the EEPROM resource, ECU will send the following Positive response:

ECU ⇒ Tester: ClearDiagnosticInformationPositiveResponse		
Byte	Contents	Value
1	ClearDiagnosticInformationPR	54 H

Note: If a failure continues to persist in the system it will be logged after the ClearDiagnosticInformation request is given.

8.1 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: ClearDiagnosticInformationNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	ClearDiagnosticInformationRequest	14 H
3	Response Code	xx H

The negative response codes (NRC) supported for the Clear Diagnostic Information are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x31	RequestOutOfRange	This NRC shall be returned if the specified groupOfDTC parameter is not supported.
0x78	ResponsePending	This NRC indicates that the request message was received correctly, and that all parameters in the request message were valid, but the action to be performed is not yet completed and the server is not yet ready to receive another request.

9 Read DTC information

The "ReadDiagnosticCodesByStatus" service will report all Diagnostic Trouble codes stored in the EEPROM. Please refer Failsafe document for the list of available DTCs.

Read DTC Information Request:

Tester ⇒ ECU : ReadDiagnosticTroubleCodesRequest		
Byte	Contents	Value
1	ReadDTC Information	19 H
2	SubFunction	xx H
3	DTCStatusAvailability Mask	xx H

Definition of Sub function:

No	Sub function Description	Sub function value
1	reportNumberOfDTCByStatusMask	01 H / 81 H
2	reportDTCbyStatusMask	02 H / 82 H

Definition of DTCStatusAvailability Mask: 0x2FH:

StatusOfDTC	Bit #	Bit State	Description
Test Failed	0	1	This bit is set to indicate a matured failing result.
TestFailedThisOperationCycle	1	1	This bit is set if the DTC was reported at least once during the current operating cycle.
PendingDTC	2	1	This bit is set and latched if a malfunction is detected during the current operating cycle.
ConfirmedDTC	3	1	This bit is set if the DTC is confirmed at least once since the last call to ClearDiagnosticInformation and if the aging criteria has not yet been satisfied.
TestNotCompletedSinceLastClear	4	0	This bit is set to indicate that the DTC test has not run to completion since the last time diagnostic information was cleared.
TestFailedSinceLastClear	5	1	This bit is set to indicate that the DTC test returned a failed result at least once since the last time diagnostic information was cleared.
TestNotCompletedThisOperationCycle	6	0	This bit is set to indicate that that the DTC test has not run to completion this operation cycle (or since the last time diagnostic information was cleared this operation cycle).
WarningIndicatorRequested	7	0	This bit is set to indicate that the Server is requesting warningIndicator to be active

9.1 reportNumberOfDTCByStatusMask

This sub function specifies that the server shall transmit to the client the number of DTCs matching the client's specified status availability mask.

reportNumberOfDTCByStatusMask Request:

Tester ⇒ ECU : ReadDiagnosticTroubleCodesRequest		
Byte	Contents	Value
1	ReadDiagnosticCodesByStatus	19 H
2	SubFunction	01 H / 81H
3	DTCStatusMask	xx H

Positive response:

ECU ⇒ Tester: ReadDiagnosticTroubleCodesPositiveResponse		
Byte	Contents	Value
1	ReadDTCInformationPR	59 H
2	SubFunction	01 H
3	DTCStatusAvailabilityMask	xx H
4	DTCFormatIdentifier (ISO_14229-1_DTCFormat)	01 H
5	DTCCountHighByte	xx H
6	DTCCountLowByte	xx H

Definition of DTC Count:

DTCCount parameter is a 2-byte unsigned numeric number containing the number of DTCs available in the server's memory based on the status mask provided by the client.

9.2 reportDTCbyStatusMask

This parameter specifies that the server shall transmit to the client a list of DTCs and corresponding Status matching a client defined status mask.

reportDTCbyStatusMask Request:

Tester ⇒ ECU : ReadDiagnosticTroubleCodesRequest		
Byte	Contents	Value
1	ReadDiagnosticCodesByStatus	19 H
2	SubFunction	02 H / 82 H
3	DTCStatusMask	xx H

Positive response:

ECU ⇒ Tester: ReadDiagnosticTroubleCodesPositiveResponse		
Byte	Contents	Value
1	ReadDTCInformation PR	59 H
2	SubFunction	02 H
3	DTCStatusAvailabilityMask	xx H
4+(n*4)	DTC n High Byte	xx H
5+(n*4)	DTC n Mid Byte	xx H
6+(n*4)	DTC n Low Byte	xx H
7+(n*4)	statusOfDTC n	xx H

Note:
n-> Number of fault records (Current project supports 10 records).
If no fault is present in the system, the following response is sent:

ECU ⇒ Tester: ReadDTCInformationPositiveResponse		
Byte	Contents	Value
1	ReadDTCInformation PR	59 H
2	SubFunction	02 H
3	DTCStatusAvailabilityMask	xx H

9.3 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: ReadDiagnosticTroubleCodesNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	ReadDTC Information	19 H
3	Response Code	xx H

The negative response codes (NRC) supported for the Read DTC Information are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x12	Sub-functionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x31	requestOutOfRange	This NRC shall be sent if the client specified a DTCMaskRecord that was not recognized by the server or an invalid DTCSnapshotRecordNumber

10 Security access

The purpose of this service is to provide a means to access data and/or diagnostic services, which have restricted access for security, emissions, or safety reasons. The security access routine is required to authenticate the tester, enabling access to protected services. It also allows the ECU to enter into Reprogramming Mode for flash process.

Security Access Request:

TESTER ⇒ ECU : SecurityAccessRequest		
Byte	Contents	Value
1	SecurityAcces Request SID	27 H
2	securityAccessType	xx H
3	securityAccessDataRecord[1]	xxH
	…	xxH
N	securityAccessDataRecord[m]	xxH

Definition of 'securityAccessType ':

No	Data Identifier	Value
1	Request Seed	01 H/ 81 H
2	Send Key	02 H/ 82 H

10.1 Request Seed

On reception of this command, ECU generates a random number known as Seed (2 byte value) and sends it as response to the tester.

Request seed command is shown below:

TESTER ⇒ ECU: Request Seed Command		
Byte	Contents	Value
1	SecurityAcces Request SID	27 H
2	securityAccessType	01 H

The seed generated shall always be a non-zero value. However, in case the security has already been unlocked, the seed returned shall be 0000H.This provides an indication to the tester that the current security level is in the unlocked state.

On receiving valid request, the ECU generates a two byte seed and sends it as a positive response as shown below:

ECU ⇒ TESTER: Send Seed Request Positive Response		
Byte	Contents	Value
1	SecurityAcces Request PR	67 H
2	securityAccessType	01 H
3	Seed (Most Significant Byte)	xx H
4	Seed (Least Significant Byte)	xx H

10.2 Send Key

In SendKey request, the tester sends a two byte authentication key back to the ECU. The securityAccessType byte will always be one more than that requested in the RequestSeed service.

TESTER ⇒ ECU: Send Key Request		
Byte	Contents	Value
1	SecurityAcces Request SID	27 H
2	securityAccessType	02 H
3	Key (Most Significant Byte)	xx H
4	Key (Least Significant Byte)	xx H

If the key sent to the ECU matches with that computed internally, a positive response is sent to indicate that the requested level of security has been unlocked.

ECU ⇒ TESTER: Send Key Positive Response		
Byte	Content	Value
1	SecurityAcces Request PR	67 H
2	securityAccessType	02 H

Note:

1. Every time an unsuccessful attempt to unlock the ECU is detected, an error counter will be incremented and can reach a maximum value of 05 in one ignition cycle. The counter shall be reset every time the security is cancelled successfully.

10.3 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ TESTER : Security Access Negative Response		
Byte	Contents	Value
1	Negative Response	7F H
2	Requested Service: “ SecurityAcces Request SID “	27 H
3	Response Code	xx H

The negative response codes (NRC) supported for the Security Access are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x12	Sub-functionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x22	Conditions not correct	This NRC shall be returned if the criteria for the request SecurityAccess are not met.
0x24	Request sequence error	Sent if the ‘sendKey’ sub-function is received without first receiving a ‘requestSeed’ request message.
0x35	Invalid Key	Sent if an expected 'sendKey' sub-function value is received and the value of the key does not match the server's internally stored/calculated key.
0x36	Exceed number of attempts	Sent if the delay timer is active due to exceeding the maximum number of allowed false access attempts.
0x37	Required time delay not expired	Sent if the delay timer is active and a request is transmitted.

11 ReadDataByIdentifier

The ReadDataByIdentifier service allows the client to request data record values from the server identified by one or more dataIdentifiers. This service will report the system information data.
The format and definition of the data record shall be vehicle manufacturer/system supplier specific, and may include analog input and output signals, digital input and output signals, internal data, and system status information, if supported by the server.

Request for ReadDataByIdentifer:

TESTER ⇒ ECU : ReadDataByLocalIdentifierRequest		
Byte	Contents	Value
1	ReadDataByIdentifier	22 H
2	Data Identifier High Byte	xx H
3	Data Identifier Low Byte	xx H

Definition of Supported Data Identifier:

No	Data Identifier description	Data Identifier Value
1	ECU Input signal	0xF102 H
2	Variant Data (Bike Model) *	0xF1FF H
3	Pressure Sensor Signals	0xF109 H
4	Reprogramming Date *	0xF199 H
5	ProcessByte *	0xF1FA H
6	Customer Part Number **	0xF187 H
7	SW Version	0xF181 H
8	SW Fingerprint **	0xF184 H
9	Vehicle Identification Number(VIN) **	0xF190 H
10	EOL-Lean Angle Data *	0xF108 H
11	EOL-Pitch Angle Data *	0xF10A H
12	EOL Mode *	0xF1F1 H
13	IMU Sensor Status	0xF107 H
14	EOL Expected Lean Angle *	0xF1F2 H
15	EOL Expected Pitch Angle *	0xF1F5 H
16	BBNumber	0xF194 H
17	Diagnostic Active Session	0xF186 H

Note:
1. In the current implementation, if the data element of the requested data identifier is empty i.e. there is no data explicitly filled into the PDM, positive response is obtained with output buffer filled with 0x30H.
2. * indicates that for the specified service, if the PDM is empty, NRC22 is triggered.
3. ** indicates that for the specified service, if the PDM is empty, the output buffer is filled with 0x00 as the mentioned services output ASCII values.

11.1 ECU Input signal

This service can be used to read the wheel speeds, the power supply voltage and the status of the valves during actuations.
This service can be read out while routine control services like Evac and Fill Repair Bleed and actuator test are being executed to check the valves being actuated.

On a valid request, the ECU will send the following Positive response:

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	02 H
4	Front Wheel Speed (1 LSB = 1 Km/h)	xx H
5	Rear Wheel Speed (1 LSB = 1 Km/h)	xx H
6	Power Supply Voltage (1 LSB = 0.078 V)	xx H
7	Valve status 1 -Bit0: Valve relay status, 1=ON -Bit1: Pump motor status, 1=ON -Bit2: Front Inlet Valve status, 1=ON -Bit3: Front Outlet Valve status, 1=ON -Bit4: Reserved -Bit5: Reserved -Bit6: Rear Inlet Valve status, 1=ON -Bit7: Rear Outlet Valve status, 1=ON	xx H
8	Valve status 2 -Bit0: Front USV status, 1=ON -Bit1: Front HSV status, 1=ON -Bit2: Rear USV status, 1=ON -Bit3: Rear HSV status, 1=ON -Bit4: Reserved -Bit5: Reserved -Bit6: Reserved -Bit7: Reserved	xx H

Note:

1. If speed greater than 0xFFH is provided as input, Bytes 4 and 5 will truncate it to a max value of FFH.

2. Front HSV and Front USV valves are absent in case of Semi-Integral system.

Definition of Valve Status

*---Relationship-Table between Inlet/Outlet Valve State and Braking Pressure

	Inlet Valve (EV)	Outlet Valve (AV)
Pressure Hold	1	0
Pressure Decrease	1	1
Pressure Increase	0	0

The valve activation is limited by overheat protection measures.

11.2 Variant Data (Bike Model)

The tester uses this request to read the variant data written in the EEPROM.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifier Response		
Byte	Content	Value
1	ReadDataByIdentifierPR	62 H
2	DataIdentifier(MSB)	F1 H
3	DataIdentifier(LSB)	FF H
4	Variant byte (Bike Model)	xx H

11.3 Pressure Sensor Signals

This service gives the pressure values at each of the four pressure sensors connected to the system.

On a valid request, the ECU will send the following positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	09 H
4	Front WC Pressure value High Byte	xx H
5	Front WC Pressure value Low Byte	xx H
6	Rear WC Pressure value High Byte	xx H
7	Rear WC Pressure value Low Byte	xx H
8	Front Wheel MC Pressure value High Byte	xx H
9	Front Wheel MC Pressure value Low Byte	xx H
10	Rear Wheel MC Pressure value High Byte	xx H
11	Rear Wheel MC Pressure value Low Byte	xx H

Pressure value: Resolution: 0.0153bar | bit signed
Value range: -42.5 … 425 bar (F527 H … 6C81 H)

11.4 Reprogramming Date

The service is used to read the date of reprogramming of ECU. On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Content	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	99 H
4	Programming Date Year (YY) MSB	xx (BCD)
5	Programming Date Month (MM)	xx (BCD)
6	Programming Date Day (DD) LSB	xx (BCD)

Note:

1. ASCII values are read from tester tool. Reprogramming Date is available only if Boot Block feature is available and is configured in boot block or programming mode.

11.5 ProcessByte

Process byte is used to check status of evacuation and fill routine.

On a valid request, the ECU will send the following positive response:

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	FA H
4	EOL Assembly process byte	xx H

Note:

1. EOL Assembly process byte: 01 (Completed: Wet state)
2. EOL Assembly process byte: 00 (Initial state: Dry state)

11.6 Customer part number

Eight byte Customer part number can be read out using this service.

On a valid request, the ECU will send the following positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	87 H
4	Customer part number (ASCII)	xx H
5		xx H
6		xx H
7		xx H
8		xx H
9		xx H
10		xx H
11		xx H

Note:

1. The ASCII values shall be read from the customer tester tool.

11.7 SW version

This service is used to read out the software version being used. Platform supports a two byte read of SWVersion to indicate the type of SW being released.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	81 H
4	SwVersion_Major	xx H
5	SwVersion_Minor	xx H

Definition of SWVersion:

SwVersion_Major : Sample release to customer (C/D sample or bugfix/running change)
SwVersion_Minor : Intermediate and internal releases

11.8 SW Fingerprint

This service is used to read the SW fingerprint.

On a valid request, the following positive response is sent:

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Content	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	84 H
4	Reprogram Part Number Position #1 (High Byte)	ASCII
5	Reprogram Part Number Position # 2	ASCII
6	Reprogram Part Number Position # 3	ASCII
7	Reprogram Part Number Position # 4	ASCII
8	Reprogram Part Number Position # 5	ASCII
9	Reprogram Part Number Position # 6	ASCII
10	Reprogram Part Number Position # 7	ASCII
11	Reprogram Part Number Position # 8	ASCII
12	Reprogram Part Number Position # 9	ASCII
13	Reprogram Part Number Position #10	ASCII
14	Reprogram Part Number Position #11	ASCII
15	Reprogram Part Number Position #12(Low Byte)	ASCII

11.9 Vehicle Identification number

The VIN number can be read using this service.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	90 H
4	VIN data (ASCII)	xx H
5		xx H
6		xx H
7		xx H
8		xx H
9		xx H
10		xx H
11		xx H
12		xx H
13		xx H
14		xx H
15		xx H
16		xx H
17		xx H
18		xx H
19		xx H
20		xx H

Note:

1. ASCII values are read from tester tool.

11.10 EOL-Lean Angle Data

This service is used to read the learnt lean angle values. This data element gets updated on running the routine "Sensor Mounting Validation Test".
The learnt value is computed by taking an average of 10 lean angle values from the sensor and compared with the expected value (stored in EEPROM).

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	08 H
4	Lean Angle Data (HB)	xx H
5	Lean Angle Data (LB)	xx H

Resolution: 1 Degree per bit

Note: Angle is read in degrees in signed word format.

SL No	Angle	HB	LB
1	Positive Values	xx	xx
2	Negative Values	Fx	xx
3	0	00	00
4	180	00	B4
5	-180	FF	4C

11.11 EOL-Pitch Angle Data

This service is used to read the learnt pitch angle values. This data element gets updated on running the routine "Sensor Mounting Validation Test".
The learnt value is computed by taking an average of 10 pitch angle values from the sensor and compared with the expected value (stored in EEPROM).

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	0A H
4	Pitch Angle Data (HB)	xx H
5	Pitch Angle Data (LB)	xx H

Resolution: 1 Degree per bit

Note: Angle is read in degrees in signed word format.

SL No	Angle	HB	LB
1	Positive Values	XX	XX
2	Negative Values	FX	XX
3	0	00	00
4	180	00	B4
5	-180	FF	4C

11.12 EOL Mode

The phase information of the EOL can be read through this request.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	F1 H
4	EOL Mode	xx H

In order for the successful running of "Sensor Mounting Validation Test" routine, this data element must be initialized with value 0x01H.

Once the routine finishes, based on the status of completion the data element EOL Mode is updated.

The table below specifies the significance of the values read using this service:

EOL Mode Byte	EOL Mode Description
0x00	Successful Completion of EOL
0x01	Phase-1 of EOL Entered
0x02	EOL has entered the lock mode

Note: 0x02H indicates that the computed value of pitch/lean angle is not within the tolerance level(10%) of the expected values.

11.13 IMU Sensor Status

This service is used to read the lean and pitch angle values along with the IMU sensor status availability.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	07 H
4	Lean Angle Low Byte	xx H
5	Lean Angle High Byte	xx H
6	Pitch Angle Low Byte	xx H
7	Pitch Angle High Byte	xx H
8	IMU Sensor Status Availability Bit0: Sensor Availability, 1= Available, 0= Not available Bit1: Reserved Bit2: Reserved Bit3: Reserved Bit4: Reserved Bit5: Reserved Bit6: Reserved Bit7: Reserved	xx H

Resolution: 1 Degree per bit

Note: Angle has to be read in degrees in signed word format.

SL No	Angle	HB	LB
1	Positive Values	xx	xx
2	Negative Values	Fx	xx
3	0	00	00
4	180	00	B4
5	-180	FF	4C

11.14 EOL Expected Lean Angle

Lean Angle data that is expected during EOL for sensor calibration is read from EEPROM.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	F2 H
4	Lean Angle High Byte	xx H
5	Lean Angle Low Byte	xx H

11.15 EOL Expected Pitch Angle

Pitch Angle data that is expected during EOL for sensor calibration is read from EEPROM.

On a valid request, the ECU will send the following Positive response,

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	F5 H
4	Pitch Angle High Byte	xx H
5	Pitch Angle Low Byte	xx H

Note: The above two values are written into the EEPROM using the WriteDataByIdentifier service.

11.16 BB Number

Five bytes of Bosch software number, stored in the flash, can be read using this service.

On a valid request, the ECU will send the following Positive response:

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse			
Byte	Contents		Value
1	ReadDataByIdentifierPR		62H
2	Data Identifier High Byte		F1 H
3	Data Identifier Low Byte		94 H
4	SW BB-Number (ASCII)	Figure 1	xx H
5		Figure 2	xx H
6		Figure 3	xx H
7		Figure 4	xx H
8		Figure 5	xx H

11.17 Diagnostic Active Session

One byte is allocated to read the currently active Diagnostic session.

On a valid request, the ECU will send the following Positive response:

ECU ⇒ TESTER: ReadDataByIdentifierPositiveResponse		
Byte	Contents	Value
1	ReadDataByIdentifierPR	62 H
2	Data Identifiers High Byte	F1 H
3	Data Identifiers Low Byte	86 H
4	DiagnosticActiveSession	xx H

Here, DiagnosticActiveSession : When control is in Application:
0x01 (Default Session)
0x03(Extended Session)
When control is in Boot:
0x81 (Default Session)
0x82(Programming Session)
0x83(Extended Session)

11.19 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: ReadDataByIdentifierNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	ReadDataByIdentifier	22 H
3	Response Code	xx H

The negative response codes (NRC) supported for the ReadDataByIdentifier are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x31	RequestOutOfRange	requested data Identifier values are not supported by the ECU
0x10	generalReject	This NRC indicates that the requested action has been rejected by the server.
0x78	requestCorrectlyReceived-ResponsePending	This NRC indicates that the request message was received correctly, but the action to be performed is not yet completed and the server is not yet ready to receive another request. As soon as the requested service has been completed, the server shall send a positive response message or negative response message with a response code different from this.

12 WriteDataByIdentifier

The 'WriteDataByIdentifier' service will store specified record data to EEPROM locations.

WriteDataByIdentifier Request:

Tester ⇒ ECU: WriteDataByIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifierRequest	2E H
2	data Identifier High Byte	xx H
3	data Identifier Low Byte	xx H
4	Data Byte 1	xx H
--	--	xx H
n+3	Data Byte n	xx H

Definition of Data Identifier:

No	Data Identifier description	Data Identifier Value
1	ProcessByte	0xF1FA H
2	VehicleIdentificationNumber (VIN)	0xF190 H
3	EOL Mode	0xF1F1 H
4	EOL Expected Lean Angle	0xF1F2 H
5	EOL Expected Pitch Angle	0xF1F5 H
6	Variant Data	0xF1FF H

12.1 ProcessByte

The process byte value can be written into the EEPROM using the below request:

Tester ⇒ ECU: WriteDataByIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifier	2EH
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	FAH
4	RecordValue	xx H

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifierPR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	FA H

12.2 Vehicle Identification number

VIN data is written using below service.

Tester ⇒ ECU: WriteDataByIdentifierRequest		
Byte	Contents	Value
1	WriteDataByIdentifier	2E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	90 H
4	VIN data (ASCII)	xx H
5		xx H
6		xx H
7		xx H
8		xx H
9		xx H
10		xx H
11		xx H
12		xx H
13		xx H
14		xx H
15		xx H
16		xx H
17		xx H
18		xx H
19		xx H
20		xx H

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifierPR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	90 H

12.3 EOL Mode

Service used to write EOL Mode in EEPROM. In order to run the EOL Routine “Sensor Mounting Validation Test”, it is essential for this data element to be initialised with the value 0x01H.
The corresponding EEPROM can be written into using the below service:

Tester ⇒ ECU: WriteDataByIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifier	2E H
2	Data Identifier High Byte	F1 H
2	Data Identifier Low Byte	F1 H
3	EOL Mode Byte	xx H

Definition of EOL Mode Byte:

EOL Mode byte	EOL Mode Description
0x00	To Exit from Lock state of EOL
0x01	To Enter Phase-1 of EOL

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifier PR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	F1 H

12.4 EOL Expected Lean Angle

This request is used to write the expected lean angle value into the corresponding data element and is essential for the "Sensor Mounting Validation Test" routine.

Tester ⇒ ECU: WriteDataByIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifier	2E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	F2 H
4	Lean Angle (HB)	xx H
5	Lean Angle (LB)	xx H

Resolution: 1 Degree per bit

Note: Angle has to be written in degrees in EEPROM in signed word format.

Sl No	Angle	HB	LB
1	Positive Values	xx	xx
2	Negative Values	Fx	xx
3	0	00	00
4	180	00	B4
5	-180	FF	4C

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifier PR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	F2 H

12.5 EOL Expected Pitch Angle

This request is used to write the expected pitch angle value into the corresponding data element and is essential for the "Sensor Mounting Validation Test" routine.

Tester ⇒ ECU: WriteDataByLocalIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifier	2E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	F5 H
4	Pitch Angle (HB)	xx H
5	Pitch Angle (LB)	xx H

Resolution: 1 Degree per bit

Note: Angle has to be written in degrees in EEPROM in signed word format.

Sl No	Angle	HB	LB
1	Positive Values	XX	XX
2	Negative Values	FX	XX
3	0	00	00
4	180	00	B4
5	-180	FF	4C

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifier PR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	F5 H

12.6 Variant Data(Bike Model)

Service used to write variant in EEPROM.

Tester ⇒ ECU: WriteDataByLocalIdentifierRequest		
Byte	Content	Value
1	WriteDataByIdentifier	2E H
2	Data Identifier High Byte	F1 H
2	Data Identifier Low Byte	FF H
3	Variant Data(Bike Model)	xx H

On successful completion of the write process, the following positive response will be sent:

ECU ⇒ Tester: WriteDataByIdentifierPositiveResponse		
Byte	Content	Value
1	WriteDataByIdentifier PR	6E H
2	Data Identifier High Byte	F1 H
3	Data Identifier Low Byte	FF H

12.7 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ TESTER : WriteDataByIdentifierNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	WriteDataByIdentifier	2E H
3	Response Code	xx H

The negative response codes (NRC) supported for the WriteDataByIdentifier are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x31	RequestOutOfRange	This NRC shall be sent if: 1.The data Identifier in the request message is not supported in the server or the data Identifier is supported for read only purpose (via ReadDataByIdentifier service); 2.Any data transmitted in the request message after the data Identifier is invalid (if applicable to the node);
0x33	SecurityAccessDenied	This NRC shall be sent if the dataIdentifier, which reference a specific address, is secured and the server is not in an unlocked state.
0x10	generalReject	This NRC indicates that the requested action has been rejected by the server.
0x78	requestCorrectlyReceived-ResponsePending	This NRC indicates that the request message was received correctly, but the action to be performed is not yet completed and the server is not yet ready to receive another request. As soon as the requested service has been completed, the server shall send a positive response message or negative response message with a response code different from this.

13 Routine Control

The RoutineControl service is used by the client to execute a defined sequence of steps and obtain any relevant results. This service will start a ROM resident routine in the ECU and is run in the Extended session.

RoutineControl Request:

Byte	Contents	Value
1	RoutineControl Request SID	31 H
2	routineControlType	xx H
3	routineIdentifierHighByte	xx H
4	routineIdentifierLowByte	xx H
…	routineControlOptionRecord	xx H

Definition of routineControlType:

No	Sub Function description	SubFunctionValue
1	Start Routine	01 H/ 81 H
2	Stop Routine	02 H/ 82 H
3	RequestRoutineResults	03 H/ 83 H

Definition of 'routinIdentifiers':

No	Routine Identifier description	routineIdentifier Value
1	Evacuation and Fill	B001 H
2	Repair Bleed	B002 H
3	Actuator test	B003 H
4	Wheel Speed Sensor Test	B004 H
5	ABS SILA Blinking Test	B005 H
6	Sensor Mounting Validation Test	B006 H

13.1 Evac and Fill

Purpose :

During the vehicle production, no air must remain in the brake system of the bike. After mounting of all hydraulic parts, an evacuation pump sucks the air out of the brake system. If the vacuum reaches a defined vacuum level, the filling machine switches off the vacuum pump and starts to fill the bike with brake fluid with a pre-pressure. A valve actuation profile ensures that no air remains in the hydraulic ABS unit.

Entry and Exit conditions:

Entry condition:

The routine will not start if any of the below specified conditions are met :

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v).
3. Other routine is running

Exit condition:

The ongoing routine comes to a stop if the below conditions are encountered:

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v)

Process:

Based on the request, Front and Rear outlet valves , Front and Rear HSV valves and the pump motor will actuate for the time specified in the request.

Note:

Speed check is done only as entry condition into extended session and will not be the entry/exit condition for routine.

Start Routine Request:

The request of Evacuation & Fill routine is shown here:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H
5	ValveControlStatus (0xFF- ON)	xx H
6	TotalEvacAndFillTime (1 LSB = 1s ; Max Limit :150s)	xx H
7	PumpControlStatus (0x00 – OFF ; 0xFF- ON)	xx H
8	MotorRunTime (Time till Start of Pump motor from Start of routine Resolution: 1sec)	xx H

Note:

1. ValveControlStatus must be made 0xFFH for valve actuations. 0x00H is an invalid value since EvacAndFill requires valve actuations.
2. Maximum TotalEvacAndFillTime should be 150 seconds and MotorRunTime must not exceed the specified TotalEvacAndFillTime.

Positive Response:

On a valid request and no failures (mentioned above), the ECU will send the following Positive response

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H

Stop Routine Request:

In order to stop the Evac and Fill process, the below request is sent:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H

Positive Response:

On a valid request, the ECU will send the following Positive response and ongoing routine is stopped.

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H

Results Routine Request:

The status of the Evac and Fill routine can be checked using the below request:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H

Positive Response:

On a valid request, a positive response indicating the state of the routine requested is sent:

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	01 H
5	Routine Status	xx H

Definition of Routine Status:

Value	Status
0x01	Routine is running
0x02	Routine has successfully finished
0x04	Routine has been stopped
0x08	Routine has been interrupted due to an error

13.2 Repair Bleeding

Purpose :

All systems that failed the on line evacuation and fill process or might be replaced by dry units, must be repair bleed with a specific procedure.

The operator has to open the bleeding screws on the wheels and the system will be filled by brake fluid.

Entry and Exit conditions:

Entry condition:

The routine will not start if any of the below specified conditions are met :

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v).
3. Other routine is running

Exit condition:

The ongoing routine comes to a stop if the below conditions are encountered:

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v)

Process:

Based on the phase requested, Front/Rear Outlet, Front/Rear HSV and the pump motor gets actuated based on the Repair Bleed pattern specified. There are a total of four phases that are defined. Repetition is possible in case of phase 2 and 4.

Note:

Speed check is done only as entry condition into extended session and will not be the entry/exit condition for routine.

Start Routine Request:

The request of Repair bleed routine is shown here:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	02 H
5	RepairBleedPhaseIdentifier	xx H
6	RepairBleedLoopRepetition	xx H

Note:

- RepairBleedPhaseIdentifier specifies the phase of the repair bleed sequence and can take values from 1 to 4.
- RepairBleedLoopRepetition specifies the repetitions required in phase 2 and 4. Also the number of repetitions must be within the range of 3 to 5 else the value gets rounded off to the nearest max/min value.

  Example: If the repetition is given as 1, it gets rounded off to 3 and if a value of 6 is given, then it gets rounded off to 5.

Positive Response:

On receiving a valid request and no failures (mentioned above), the ECU will send the following Positive response:

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	02 H

Stop Routine Request:

Request to stop Repair bleed routine is as given below:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	02 H

Positive Response:

On receiving valid stop routine request, below response is sent

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	02 H

Routine Results Request:

The status of the Repair Bleed routine can be checked using the request result as shown below:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H

4	routine Identifier Low Byte	02 H

Positive Response:

On receiving a valid RequestRoutineResults request, below response is sent

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	02 H
5	Routine Status	xx H

Definition of Routine Status:

Value	Status
0x01	Routine is running
0x02	Routine has successfully finished
0x04	Routine has been stopped
0x08	Routine has been interrupted due to an error

13.3 Actuator Test

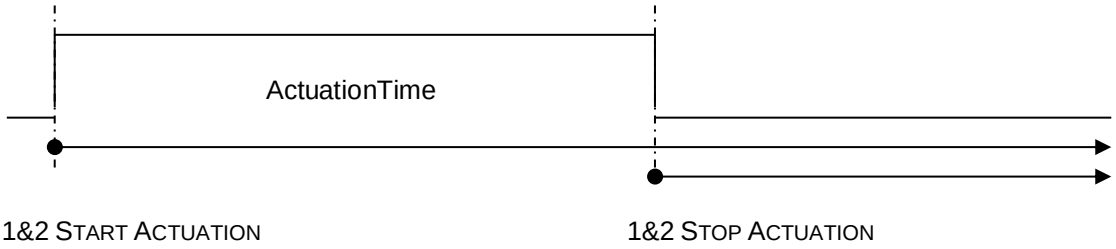
Purpose:

The Actuator Test is a routine that gives the possibility to activate dedicated actuators (e.g. inlet valve / outlet valve). For each actuator the actuator number and the actuator value needs to be set. This routine allows for actuation of valves and pump motor.

The normal use case of the Actuator test is to enable Actuators 1 and 2 for a certain time equal to the Actuation time .After this time the same actuators are switched off in the request by requesting OFF through 3/4.

The software has a self-protection mechanism (Diagnostic Hydraulic Protection- DHP) which will set a failure when the time limit is exceeded, ensuring that the valves and motors are not damaged.
The actuator numbers and the values to enable / disable them are available in the diagnostic request.

Refer Appendix A for the list of actuators available.



Entry and Exit conditions:

Entry condition:

The routine will not start if any of the below specified conditions are met :

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v).
3. Other routine is running

Exit condition:

The ongoing routine comes to a stop if the below conditions are encountered:

1. Failures that shut down the hydraulic (refer Fail-Safe specification).
2. Over Voltage (16.5v) and Hard Under voltage (8.5v)

Note:
Speed check is done only as entry condition into extended session and will not be the entry/exit condition for routine.

Start RoutineRequest:

The Request of Actuator routine is shown here:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H
5-6	1.Actuation Value	xxxx H
7-8	1.Actuator Number	xxxx H
9-10	2.Actuation Value	xxxx H
11-12	2.Actuator Number	xxxx H
13-14	Actuation Time (1LSB = 10ms, Max limit: 30s)	xxxx H
15-16	3.Actuation Value	xxxx H
17-18	3.Actuator Number	xxxx H
19-20	4.Actuation Value	xxxx H
21-22	4.Actuator Number	xxxx H

Positive Response:

On a valid request and no failures (mentioned above), the ECU will send the following Positive response:

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H

Stop Routine Request:

The Actuator Test can be stopped in between by a stop routine request.

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H

Positive Response:

On receiving valid stop routine request, below response is sent

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H

Routine Results Request:

The status of the actuator test can be checked using the request below:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest PR	31 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H

Positive Response:

On receiving valid RequestRoutineResults request, below response is sent

ECU ⇒ Tester: RoutineControlPositiveResponse		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	03 H
5	Routine Status	xx H
6	Result Byte : Actuator 1	xx H
7	Result Byte : Actuator 2	xx H
8	Result Byte : Actuator 3	xx H
9	Result Byte : Actuator 4	xx H

Definition of Routine Status:

Value	Status
0x01	Routine is running
0x02	Routine has successfully finished
0x04	Routine has been stopped
0x08	Routine has been interrupted due to an error

Note:

A DHP error is set if the actuation limit exceeds the permitted level mentioned below:

Component	Maximum Actuation Time(in seconds)
Inlet Valve - EV	60
Outlet Valve -AV	30
USV	60
HSV	60
Pump motor	150

The Valve/Pump time limitation up counter starts when there is an actual requested actuation value (i.e. greater than 0). Once the defined time limit is reached, the protection state will be set to Valve/Pump failed and a failure information is also set accordingly till the protection state is set.

It takes twice the time for the down counter as that of the up counter to release the protection state after a failure For example, for an EV failure, it takes 60s to set the protection state and 120s to release it.

Any actuation request during the protection state set time will be suppressed and hence will not be processed.

13.4 Wheel Speed Sensor Test

Purpose:

This test can be used to check correlation of the Wheel Speed Sensors (connection of the sensors to the corresponding wheel) and for checking signal quality.

Entry and Exit conditions:

Entry condition:

The routine will not start if any of the below specified conditions are met:

1. Failure pertaining to wheel speed sensors
2. Other routine is running

Exit condition:

The ongoing routine comes to a stop if the below conditions are encountered:

1. If a WSS related failure is set

Process:

The Wheel Speed Sensor test routine checks all the wheel speeds for a specified time and provides the maximum and minimum wheel speed in the given interval.
The test duration is chosen to be equivalent to one complete wheel revolution.

Start Routine Request:

The request of WSS routine is shown here:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H
5	Execution Time (HB)	xx H
6	Execution Time (LB)	xx H

Execution Time: 1 LSB = 10 ms

Positive Response:

On a valid request and no failures (mentioned above), the ECU will send the following positive response:

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H

Stop Routine Request:

The request to stop WSS routine is shown here:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H

Positive Response:

On receiving valid stop routine request, below response is sent:

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	02 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H

Request Routine Results:

WSS routine status can be checked using the below request:

Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H

Positive Response:

On receiving a valid RequestRoutineResults request, the status of the WSS routine will be received as shown below:

ECU ⇒ Tester: Routine Control Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	03 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	04 H
5	Routine Status	xx H
6	Max. Wheel speed front(HB)	xx H
7	Max. Wheel speed front(LB)	xx H
8	Min. Wheel speed front(HB)	xx H
9	Min. Wheel speed front(LB)	xx H
10	Max. Wheel speed Rear(HB)	xx H
11	Max. Wheel speed Rear(LB)	xx H
12	Min. Wheel speed Rear(HB)	xx H
13	Min. Wheel speed Rear(LB)	xx H

Definition of Routine Status:

Value	Status
0x01	Routine is running
0x02	Routine has successfully finished
0x04	Routine has been stopped
0x08	Routine has been interrupted due to an error

Note:

1. Resolution of speed v = 0.05625 Km/h
2. If the routine is stopped by giving the stop routine command (31 02 B0 04), the request result response will contain the speeds calculated until the time the routine was stopped.
3. If the routine has been interrupted due to an error (say, WSS failure), then the request results will respond with FFFFH for the speeds to indicate that the routine was stopped due to a failure in the system.

13.5 ABS SILA Blinking Test

Purpose:

This test is for Original Equipment Manufacturer (OEM) to check the ABS SILA at their End Of Line (EOL). This test routine blinks the SILA based on a specific pattern.

Entry condition:

If any of the following conditions are met, the routine will not start and ECU will send the ConditionsNotCorrect NRC (22H):

1. Any current failure
2. Other routine is running

Start Routine Request:

The ABS Sila Blink Test can be requested using the below command:

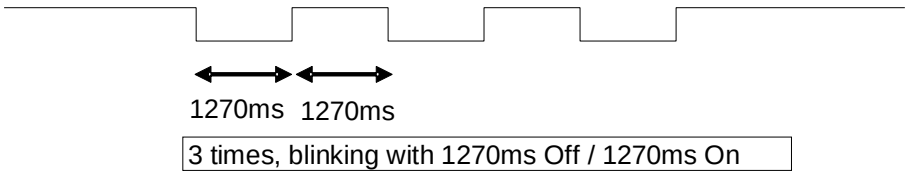
Tester ⇒ ECU: Routine Control Request		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	05 H

Positive Response

On a valid request and no failures (mentioned above), the ECU will send the following Positive response :

ECU ⇒ Tester: Routine Control Positive Response		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	routineControlType	01 H
3	routine Identifier High Byte	B0 H
4	routine Identifier Low Byte	05 H

The SILA blinking pattern is as shown below. Note that this pattern starts with “ON” because this routine is done in the extended session where the SILA is already turned ON.



Note:

1. Stop and Result routines are not applicable for ABS Sila Test
2. Routine is valid for hardwired SILA and not for SILA on CAN

13.6 Sensor Mounting Validation Test

Purpose :

This routine is done to validate the mounting of the IMU sensor onto the bike. A comparison between the expected values of lean and pitch angles and the learned values are done to check if the values are within a tolerance of 10%.

The routine must be executed only after the PDMs corresponding to EOL mode, EOL Expected lean Angle and EOL expected Pitch Angle are initialised using the WriteDataByIdentifier Service (Refer section 12.3,12.4 and 12.5).

Entry Condition:

The routine will not start if any of the below specified conditions are met :

1. Any current failure relating to IMU sensor
2. Other routine is running
3. If EOL Mode is not initialised to 1

Start RoutineRequest

The Sensor mounting test can be requested using the below service:

Tester ⇒ ECU: StartRoutineByLocalIdentifierRequest		
Byte	Content	Value
1	RoutineControlRequest SID	31 H
2	routineControlType	01 H
3	RoutineLocal Identifier	B0 H
4	EOLRoutineIdentifier	06 H
5	EOL Phase	01 H

Process:

The above request is to execute the “EOL phase 1 “.In this process, ECU evaluates the lean angle and pitch angle for 10 samples and the average will be compared against a predefined value (expected lean and pitch angle). If the calculated lean angle and pitch angle is not within range of the expected lean angle and pitch, the EOL will be locked.

On completion of Phase 1, the system exits from EOL with below positive response. If the routine has been executed successfully, 0x00H is written into EOL Mode else a value of 0x02H is written.

ECU ⇒ Tester: StartRoutineByLocalIdentifierPositiveResponse		
Byte	Content	Value
1	RoutineControlRequest PR	71 H
2	RoutineLocalIdentifier	B0 H
3	EOLRoutineIdentifier	06 H

Note: Stop and Result routines are not applicable for Sensor Mounting Validation Test.

13.7 Negative response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: RoutineControlNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	Routine control Request	31 H
3	Response Code	xx H

The negative response codes (NRC) supported for the Routine Control Services are listed in the table below. The circumstances under which they are sent by the ECU is given alongside.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the requested sub-function is either generally not supported or is not supported for the requested Routine Identifier.
0x22	ConditionsNotCorrect	This NRC shall be returned if the entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord
0x24	RequestSequenceError	1. The routine is not currently active when the 'stop Routine' sub-function is received. 2. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x7F	ServiceNotSupportedInActiveSession	This NRC indicates that the requested action will not be taken because the server ssdoes not support the requested service in the session currently active.
0x33	SecurityAccessDenied	This NRC shall be sent if the dataIdentifier, which reference a specific address, is secured and the server is not in an unlocked state.

14 Communication Control

The tester can use this service to control communication on the CAN bus. The purpose of communication control is to switch on/off the transmission and/or the reception of certain messages of server(s).

CommunicationControl Request:

Tester ⇒ ECU: CommunicationControlRequest		
Byte	Content	Value
1	CommunicationControlRequest SID	28 H
2	SubFunction	xx H
3	communicationType	xx H

Definition of SubFunction:

Sl no	SubFunction Value	SubFunction	Description
1	00 H	enableRxAndTx	This value indicates that the reception and transmission of messages shall be enabled for the specified communicationType. suppressPosRspMsgIndicationBit=FALSE
2	01 H	enableRxAndDisableTx	This value indicates that the reception of messages shall be enabled and the transmission shall be disabled for the specified communicationType. suppressPosRspMsgIndicationBit=FALSE
3	80 H	enableRxAndTx	This value indicates that the reception and transmission of messages shall be enabled for the specified communicationType. suppressPosRspMsgIndicationBit=TRUE
4	81 H	enableRxAndDisableTx	This value indicates that the reception of messages shall be enabled and the transmission shall be disabled for the specified communicationType. suppressPosRspMsgIndicationBit=TRUE

Definition of communicationType:

Sl no.	communicationType Value	communicationType	Description
1	0x00 H	ISOSAEReserved	-
2	0x01 H	normalCommunicationMessages	This value references all application-related communication (interapplication signal exchange between multiple in-vehicle servers).
3	0x02 H	networkManagementCommunicationMessages	This value references all network management related communication.
4	0x03 H	networkManagementCommunicationMessages and normalCommunicationMessages	This value references all network management and application-related communication.

Note:

- For the current project implementation, there is no network management supported, hence communication is enabled/disabled for communicationType 0x01H and 0x03H only.
- Session change from extended to default session will enable reception and transmission of all messages.

Positive Response:

On a valid request, the ECU will respond with following positive responses:

ECU ⇒ Tester: CommunicationControlPositiveResponse		
Byte	Content	Value
1	CommunicationControl Request PR	68 H
2	SubFunction	xx H

If suppressPosRspMsgIndicationBit=TRUE, no response is sent.

14.1 Negative response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: RoutineControlNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	Communication Control	28 H
3	Response Code	xx H

The table below indicates the NRCs supported for CommunicationControl and the circumstances under which they occur:

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.
0x31	requestOutOfRange	This NRC shall be returned if CommunicationType is not in requested range.
0x7F	serviceNotSupportedInActiveSession	This NRC indicates that the requested action will not be taken because the server does not support the requested service in the session currently active. Occurs when service is requested in default session.

15 ControlDTCSetting

The tester can deactivate and activate the storage of DTCs using this service. This service shall be used by a client to stop or resume the updating of DTC status bits in the server(s).

ControlDTCSetting Request:

Tester ⇒ ECU: ControlDTCSettingRequest		
Byte	Contents	Value
1	ControlDTCSetting	85 H
2	SubFunction	xx H

Definition of SubFunction:

Sl no	SubFunction Value	Description
1	01 H	DTC storage is enabled; suppressPosRspMsgIndicationBit=FALSE
2	02 H	DTC storage is disabled; suppressPosRspMsgIndicationBit=FALSE
3	81 H	DTC storage is enabled; suppressPosRspMsgIndicationBit=TRUE
4	82 H	DTC storage is disabled; suppressPosRspMsgIndicationBit=TRUE

Positive Response:

On valid request, the ECU will respond with following positive response:

ECU ⇒ Tester: ControlDTCSettingResponse		
Byte	Content	Value
1	ControlDTCSetting PR	C5 H
2	subFunction	xx H

If suppressPosRspMsgIndicationBit=TRUE, no response is sent.

15.1 Negative response

Negative response code will be sent in the format as indicated below:

ECU ⇒ Tester: ControlDTCNegativeResponse		
Byte	Contents	Value
1	Negative Response	7F H
2	Control DTC setting	85 H
3	Response Code	xx H

The table below indicates the NRCs supported for ControlDTCSetting and the circumstances under which they occur:

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.
0x7F	serviceNotSupportedInActiveSession	This NRC indicates that the requested action will not be taken because the server does not support the requested service in the session currently active. Occurs when service is requested in default session.

16 Tester Present

The "Tester Present" service can be used to maintain a diagnosis session (other than the default Session), when no other service is being executed. This service is used to indicate to a server (or servers) that a client is still connected to the vehicle and that certain diagnostic services and/or communication that have been previously activated are to remain active.

The ECU will reset S3 server to monitor S3 max again after sending the positive response.

Tester Present Request:

TESTER ⇒ ECU : TesterPresentRequest		
Byte	Contents	Value
1	TesterPresentRequest	3E H
2	ZeroSubFunction	xx H

Definition of ZeroSubFunction :

suppressPosRspMsgIndicationBit (7th Bit of ZeroSubFunction)	Value	Description
FALSE	00 H	Positive response is obtained
TRUE	80 H	Positive response is suppressed

Positive Response:

On valid request, the ECU will respond with the following positive response:

TESTER ⇒ ECU : TesterPresentPositiveResponse		
Byte	Contents	Value
1	Tester PresentPR	7E H
2	ZeroSubFunction	00 H

16.1 Negative Response

Negative response code will be sent in the format as indicated below:

ECU ⇒ TESTER : Tester Present Negative Response		
Byte	Contents	Value
1	Negative Response	7F H
2	Tester Present	3E H
3	Response Code	xx H

The table below indicates the NRCs supported for Tester Present and the circumstances under which they occur:

Response Code	Description	Reason
0x13	IncorrectMessageLength Or Invalid Format	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the sub-function parameter is not supported.

17 Flash Download

This section describes the format of all the requests sent by the tester during the reprogramming session and the corresponding responses sent by the ECU. The ECU can be flashed a maximum of 90 times. The entry to reprogramming Session is possible by sending the Diagnostic Session command 10 02 after successful authentication. The Tester must send the command in the same sequence as specified in section 17.3. Any deviation in the sequence (optional service Write flash programming information can be omitted), will keep the ECU in reprogramming mode and the whole flash process has to be repeated.

Note: The P3 timer shall be disabled on receiving this command and a positive response sent.

17.1 Erasure Routines

Erasure Request Command Format

The tester sends this command once to initiate the erasure process:

TESTER ⇒ ECU : Erasure Request		
Byte	Contents	Value
1	Routine control request	31 H
2	Start Routine	01 H
3	routine Identifier High Byte	FF H
4	routine Identifier Low Byte	00 H
5	Address and Length Format Identifier	01H
6	BlockId	01H

AddressNLengthFormatID - 0x01 It's the Size of BlockId in Request (1Byte)
BlockId - 0x01 BlockID of the Application software

Erasure Request Positive Response Format

The following response will be sent after the erase process is completed (Successful or unsuccessful).

ECU ⇒ TESTER: Erasure Request Positive Response		
Byte	Content	Value
1	Routine control service Id	71 H
2	Sub Function	01 H
3	routine Identifier High Byte	FF H
4	routine Identifier Low Byte	00 H
5	Routine Results	xxH

Routine Result- 0x00- Erase Succeeded
0x01- Erase Failed

Erasure Request Negative Response Format

Negative response code will be in below format

ECU ⇒ Tester: RoutineControlNegativeResponse		
Byte	Contents	Value
1	Negative Response	7FH
2	Routine control Request	31H
3	Response Code	xx H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the requested sub-function is either generally not supported or is not supported for the requested Routine Identifier.
0x22	ConditionsNotCorrect	This NRC shall be returned if the Entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord
0x24	RequestSequenceError	1.The routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x7E	Sub function not supported in active session to exist in another session.	This code indicates that the requested action will not be performed because the device does not support the requested sub-function in the currently active session. This response code should only be used when the requested sub-function is known

0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.

17.2 Flash Routine

17.2.1 Request Download

Request Download Command Format

This service is used by the diagnostic tester to initiate the download procedure.

TESTER ⇒ ECU: Request Download Request		
Byte	Content	Value
1	Request Download	34 H
2	Data format Identifier	00H
3	AddressAndLengthFormatIdentifier	41H
4	Block ID	01H
5	Length [L1] MS byte	YY
6	Length [L2]	YY
7	Length [L3]	YY
8	Length [L4] LS byte	YY

Note:

1.AddressAndLengthFormatIdentifier = 0x41.
Higher Nibble specifies Size of Download Length in Request, 4 byte is recommended.
Lower Nibble specifies Size of Block Index in Request (currently only value 1 supported)

2.BlockID= Value range 0x00…..0Xff. Application software- 0x01

3.Data format identifier 00H- where higher nibble specifies Compression and lower nibble specifies Encryption

4.YYYYYYYY is the length of the FSW in HEX and this value depends on the controller chosen.

Request Download Positive Response

ECU ⇒ TESTER: Request Download Positive Response		
Byte	Content	Value
1	Request Download	74 H
2	Length Format Identifier	20 H
2	Maximum block Length (high byte)	0F H
3	Maximum block Length (low byte)	F2 H

Request Download Negative Response

ECU ⇒ TESTER: Request Download Negative Response		
Byte	Content	Value
1	Negative Response	7F H
2	Requested Service: “Start Routine By Local Identifier“	34 H
3	Response Code	XX H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x22	ConditionsNotCorrect	This NRC shall be returned if the Entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord
0x24	RequestSequenceError	1.the routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.

17.3 Transfer Data

Transfer Data Command Format

The actual program codes are sent in this command. The command is sent repeatedly until all the program codes are sent.

TESTER ⇒ ECU: Transfer Data Request		
Byte	Content	Value
1	Transfer Data	36 H
2	Block sequence number	01 H
3	Transfer data byte 1	xx H
...	...	...
(MBL)	Transfer data Byte..N(MBL)	xx H

Transfer Data Positive Response

A successful transfer is acknowledged by a positive response:

ECU ⇒ TESTER: Transfer Data Positive Response		
Byte	Content	Word
1	Transfer Data	76 H
2	Block sequence number	xxH

Definition of BlockSequenceNumber:
One byte, range 0x00 – 0xFF
Initvalue is: 0x01
increased in every transfer data request
overrun at 0xFF (next value is 0x00)

Transfer Data Negative Response

ECU ⇒ TESTER: Transfer Data Negative Response		
Byte	Content	Value
1	Negative Response	7F H
2	Requested Service: "Start Routine By Local Identifier"	36 H
3	Response Code	XX H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.

0x71	Transfer data suspended	The requested action will not be taken because the number of bytes sent by the tester is not divisible by 4
0x22	ConditionsNotCorrect	This NRC shall be returned if the Entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord
0x24	RequestSequenceError	1.the routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.
0x72	General programming failure	The requested action will not be taken because the flash drivers have indicated that the flash could not be programmed (because of a HW error)
0x73	Wrong block sequence counter	The requested action will not be taken because the sequence counter in the transfer data request is not correct

17.3.1 Request Transfer Exit

Request Transfer Exit Command Format

This service is used to indicate that a data transfer has been completed.

TESTER ⇒ ECU: Request Transfer Exit Request		
Byte	Content	Word
1	Request Transfer Exit	37 H

Request Transfer Exit Positive Response:

ECU ⇒ TESTER: Request Transfer Exit Positive Response		
Byte	Content	Word
1	Request Transfer Exit	77 H

Request Transfer Exit Negative Response:

ECU ⇒ TESTER: Request Transfer Exit Negative Response		
Byte	Content	Value
1	Negative Response	7F H
2	Requested Service: "Start Routine By Local Identifier"	37 H
3	Response Code	XX H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLength OrInvalidFormat	This NRC shall be sent if the length of the message is wrong.

0x24	RequestSequenceError	1.the routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.

17.3.2 Memory Check

The check routine is used to check if the down-load process is completed successfully

17.3.3 Memory Check Routine

Memory Check Routine Command Format

TESTER ⇒ ECU : Erasure Request		
Byte	Contents	Value
1	Routine control request	31 H
2	Start Routine	01 H
3	routine Identifier High Byte	02 H
4	routine Identifier Low Byte	02 H
5	AddressLengthFormatID	11H
6	BlockID	01H
7	Length of Checksum	04H
8	Checksum byte 1	xxH
9	Checksum byte 2	xxH
10	Checksum byte 3	xxH
11	Checksum byte 4	xxH

Memory Check Routine Positive Response

ECU ⇒ TESTER: Memory Check Routine Positive Response		
Byte	Content	Value
1	Routine control response	71 H
2	Start routine	01H
2	routine Identifier High Byte	02H
3	routine Identifier Low Byte	02H
4	Response Code	XX H

Response Code	Description	Reason
00 H	Check Successfully Completed.	
01 H	Check Unsuccessfully Completed.	

17.3.4 Memory Check Routine Negative Response

If the tester uses any other value in the request, the ECU will respond with the following Negative Response.

ECU ⇒ TESTER: Memory Check Routine Negative Response		
Byte	Content	Value
1	Negative Response	7F H
2	Requested Service: "Start Routine By Local Identifier"	31 H
3	Response Code	XX H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the requested sub-function is either generally not supported or is not supported for the requested Routine Identifier.
0x22	ConditionsNotCorrect	This NRC shall be returned if the Entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord

0x24	RequestSequenceError	1.the routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received
0x7E	Sub function not supported in active session	The requested action will not be taken because the device does not support the requested sub-function in the current diagnostic session
0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.

17.3.5 Check Programming Dependency

17.3.5.1 Check Programming Dependency Command Format

This service is for successful authentication after the completion of the flash process.
Signature check shall be performed to authenticate the software.

Tester ⇒ ECU: StartRoutineByLocalIdentifierRequest		
Byte	Contents	Value
1	StartRoutineByLocalIdentifier	31 H
2	Start Routine	01 H
3	routine Identifier High Byte	FF H
4	routine Identifier low Byte	01 H

Signature Routine Positive Response

ECU ⇒ Tester: StartRoutineByLocalIdentifierPositiveResponse		
Byte	Contents	Value
1	StartRoutineByLocalIdentifierPR	71 H
2	Start Routine	01 H
3	routine Identifier High Byte	FFH
4	routine Identifier High Byte	01H
3	Test status	xx H

Response Code	Description	Reason
00 H	Signature OK	
01 H	Signature Check failure.	

Positive response for this service is a must to proceed further for the normal ABS operation.

17.3.5.2 Signature Check Negative Response

If the tester uses any other value in the request, the ECU will respond with the following Negative Response.

ECU ⇒ TESTER: Signature Routine Negative Response		
Byte	Content	Value
1	Negative Response	7F H
2	Requested Service: “Start Routine By Local Identifier“	31 H
3	Response Code	XX H

If the signature test isn’t finished yet, the following negative response will be made:

ECU ⇒ Tester: Signature Routine Multi Response		
Byte	Contents	Value
1	Negative Response	7F H
2	Requested Service: „StartRoutineByLocalIdentifier“	31 H
3	Signature Check Ongoing	78 H

The following negative response codes shall be implemented for this service. The circumstances under which each response code would occur are mentioned below.

Response Code	Description	Reason
0x13	IncorrectMessageLengthOrInvalidFormat	This NRC shall be sent if the length of the message is wrong.
0x12	sub-functionNotSupported	This NRC shall be sent if the requested sub-function is either generally not supported or is not supported for the requested Routine Identifier.
0x22	ConditionsNotCorrect	This NRC shall be returned if the Entry criteria for the request Routine Control are not met.
0x31	RequestOutOfRange	Invalid routine Identifier and routineControlOptionRecord
0x24	RequestSequenceError	1.the routine is currently active and cannot be restarted when the 'Start Routine' sub-function is received. 2. The routine is not currently active when the 'stop Routine' sub-function Is received. 3. Routine results are not available when the ‘requestRoutineResults’ sub-function is received

0x72	General programming failure	The requested action will not be taken because the flash drivers have indicated that the flash could not be programmed (because of a HW error)
0x33	SecurityAccessDenied	This NRC shall be sent if the data Identifier, which reference a specific address, is secured And the server is not in an unlocked state.

17.4 Flashing Sequences

The standard Flash sequence is as shown below.

Step	Action	Description
Reprograming preparation		
1	Diagnostic Session Control	Extended Session
2	Control DTC Setting	Turn OFF DTC settings
3	Communication Control	Disable non-diag Communication
Reprograming sequence		
4	Diagnostic Session Control	Programming Session
5	Seed Request	Send Seed Request Command Format
6	Security Cancellation Request.	Security Cancellation Request Command Format
7	Read Data Identifiers	Read Part no
8	Routine Control	Erasure Request
9	Request Download command	Request Download
10	Transfer Data Command till all the program codes	Transfer Data
11	Request Transfer Exit Command	Request Transfer Exit
12	Routine Control	Memory Check Routine
13	Routine Control	Check Programming Dependencies
14	Write Flash Reprogramming Information.	Programming date and Fingerprint information
15	ECU Reset Command.	ECU Reset
PostReprograming sequence		
16	Diagnostic Session Control	Extended Session
17	Communication Control	Enable non-diag Communication
18	Control DTC Setting	Turn ON DTC settings

18 Appendix A: List of Actuators

Actuator No. (Hex)	Content	*	**	Actuation Value
0000	No activation			
0022	Pump-Motor	d	i/o	0xFFFF: on 0x0000: off
0030	EV front	d	i/o	0xFFFF: on 0x0000: off
0032	AV front	d	i/o	0xFFFF: on 0x0000: off
0038	EV rear	d	i/o	0xFFFF: on 0x0000: off
003A	AV rear	d	i/o	0xFFFF: on 0x0000: off
0050	USV Rear	d	i/o	0xFFFF: on 0x0000: off
0054	HSV Rear	d	i/o	0xFFFF: on 0x0000: off
004E	USV Front	d	i/o	0xFFFF: on 0x0000: off
0052	HSV Front	d	i/o	0xFFFF: on 0x0000: off
0064	Disable VR-shut down	d	o	0xFFFF: Disable shut down 0x0000: Enable shut down

Where,
*: a = Analog Value; d = Digital Value
**: i = input signal; o = output signal; i/o = input and output signal